

Development Guidelines

Version: 1.0

Purpose

This document defines the mandatory development workflow for Trading Platform Pro. The objective is to ensure consistent implementation, high software quality and long-term maintainability.

Development Philosophy

Every change should make the project better.

Prefer:

- simplicity
- readability
- maintainability
- consistency

Avoid:

- unnecessary complexity
- speculative implementations
- duplicated code
- hidden side effects

Standard Development Workflow

Every implementation follows this sequence:

1. Analyze existing implementation
2. Understand surrounding architecture
3. Design the smallest meaningful change
4. Implement
5. Execute tests
6. Verify code quality
7. Commit
8. Push
9. Update documentation (if required)

No step should be skipped.

Before Writing Code

Always:

- understand the requirement
- inspect the affected files
- understand existing architecture
- identify dependencies
- preserve public APIs whenever possible

Implementation Rules

Every implementation should:

- solve exactly one problem
- minimize side effects
- preserve architectural boundaries
- remain independently testable
- avoid unnecessary abstractions

Code Quality

Mandatory:

- Python 3.13
- UTF-8
- ``from __future__ import annotations``
- Full type hints
- Ruff compliant
- Clear naming
- Small functions
- Small classes

Refactoring

Refactoring should:

- improve readability
- reduce complexity
- remove duplication
- preserve behaviour

Large refactorings should be split into multiple small commits.

Testing

Every code change requires testing.

Preferred order:

1. Unit Tests
2. Integration Tests
3. System Tests

All tests must pass before committing.

Git Workflow

Preferred sequence:

```

Implement

↓

pytest

↓

git add

↓

git commit

↓

git push

```

Commits should be:

- small
- atomic
- descriptive

Documentation

Documentation is part of the implementation.

Update documentation whenever:

- architecture changes

- behaviour changes
- APIs change
- workflows change

Architecture Decision Process

Before implementing architectural changes:

1. Verify whether an Architecture Decision Record (ADR) already exists.
2. Evaluate the impact on existing modules.
3. Preserve architectural boundaries.
4. Update the corresponding architecture documentation if required.

Architectural decisions should remain consistent with the Software Architecture Document (SAD).

Definition of Analysis

Implementation must never start before:

- understanding the business requirement
- reading the affected source files
- identifying dependencies
- reviewing related documentation
- evaluating possible side effects

Analysis is mandatory.

Definition of Review

Every implementation should be reviewed for:

- correctness
- architecture compliance
- coding standards
- documentation updates
- test coverage
- maintainability

Backward Compatibility

Unless explicitly approved:

- preserve public interfaces
- avoid breaking configuration
- maintain existing behaviour
- keep migration effort minimal

Breaking changes require documentation.

Security Review

Before completion verify:

- no secrets committed
- no credentials exposed
- configuration remains secure
- logging does not expose sensitive information

Performance Review

Evaluate whether the implementation:

- introduces unnecessary allocations
- increases startup time

- increases runtime complexity
 - performs redundant operations
- Optimize only when measurable.

Release Workflow

Before releasing:

1. Tests pass.
2. Documentation updated.
3. Version updated (if applicable).
4. Changelog updated.
5. Review completed.
6. Changes committed.
7. Changes pushed.

Pull Request Checklist

Before merging:

- Tests passed
- Ruff passed
- Architecture respected
- Documentation updated
- No unnecessary dependencies
- No duplicated logic

Definition of Done

A task is complete only if:

- implementation finished
- tests passed
- lint passed
- documentation updated
- committed
- pushed

Related Documents

- AGENTS.md
- Coding_Standards.md
- Git_Workflow.md
- Testing_Strategy.md
- Architecture.md